Experiment-2.3

Student Name: Gagnesh Kakkar
UID: 23BCS11196
Branch: B.E-C.S.E
Section/Group: 23KRG-2B
Semester: 5th
Date of Performance: 06/10/2025
Subject Name: DAA
Subject Code: 23CSH-301

1. **Aim:** Develop a program and analyze complexity to implement 0-1 Knapsack using Dynamic Programming.

2. **Objective:** To implement 0-1 Knapsack using Dynamic Programming.

3. **Input/Apparatus Used:** In order to fill knapsack, we can pick only complete item not in fraction.

4. **Procedure/Algorithm: Pseudocode:**

In the Dynamic programming we will work considering the same cases as mentioned in the recursive approach. In a DP[][] table let‟s consider all the possible weights from „1‟ to „W‟ as the

- Fill „wi‟ in the given column.
- Do not fill „wi‟ in the given column.

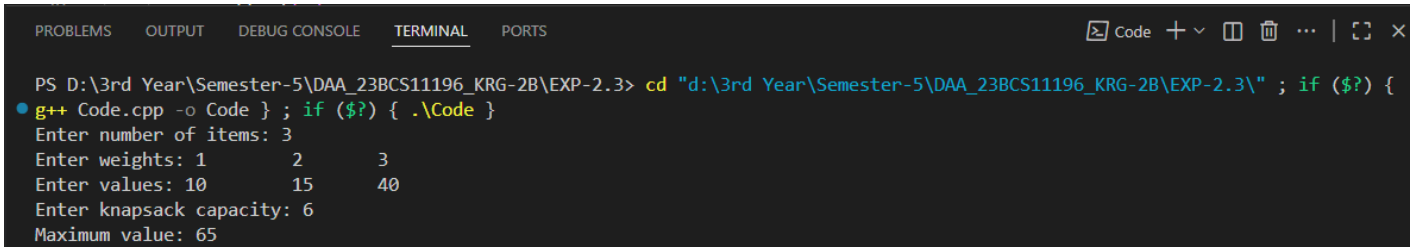
Now we have to take a maximum of these two possibilities, formally if we do not fill „ith‟weight in „jth‟ column then DP[i][j] state will be same as DP[i-1][j] but if we fill the weight, DP[i][j] will be equal to the value of „wi‟+ value of the column weighing „j-wi‟ in the previous row. So we take the maximum of these two possibilities to fill the current state.

5. Code:

Code.cpp

```cpp
#include <iostream>
using namespace std;

int knapsack(int W, int wt[], int val[], int n) {
    int dp[n+1][W+1];
    for(int i=0;i<=n;i++){
        for(int j=0;j<=W;j++){
            if(i==0 || j==0)
                dp[i][j] = 0;
            else if(wt[i-1] <= j)
                dp[i][j] = max(val[i-1] + dp[i-1][j-wt[i-1]], dp[i-1][j]);
            else
                dp[i][j] = dp[i-1][j];
        }
    }
    return dp[n][W];
}

int main() {
    int n, W;
    cout << "Enter number of items: ";
    cin >> n;

    int wt[n], val[n];

    cout << "Enter weights: ";
    for(int i=0;i<n;i++)
        cin >> wt[i];

    cout << "Enter values: ";
    for(int i=0;i<n;i++)
        cin >> val[i];

    cout << "Enter knapsack capacity: ";
    cin >> W;

    cout << "Maximum value: " << knapsack(W, wt, val, n);
}
```

6. Output:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Code
PS D:\3rd Year\Semester-5\DAA_23BCS11196_KRG-2B\EXP-2.3> cd "d:\3rd Year\Semester-5\DAA_23BCS11196_KRG-2B\EXP-2.3\" ; if ($?) {
g++ Code.cpp -o Code } ; if ($?) { .\Code }
Enter number of items: 3
Enter weights: 1 2 3
Enter values: 10 15 40
Enter knapsack capacity: 6
Maximum value: 65